

### ⓘ Note

If you have a desktop application that uses the **Desktop Bridge**, you may need to add additional code not shown in these examples to configure the **StoreContext** object. For more information, see [Using the StoreContext class in a desktop application that uses the Desktop Bridge](#).

## Get info for the current app

To get Store product info about the current app, use the [GetStoreProductForCurrentAppAsync](#) method. This is an asynchronous method that returns a [StoreProduct](#) object that you can use to get info such as the price.

C#

```
private StoreContext context = null;

public async void GetAppInfo()
{
    if (context == null)
    {
        context = StoreContext.GetDefault();
        // If your app is a desktop app that uses the Desktop Bridge, you
        // may need additional code to configure the StoreContext object.
        // For more info, see https://aka.ms/storecontext-for-desktop.
    }

    // Get app store product details. Because this might take several
    // moments,
    // display a ProgressRing during the operation.
    workingProgressRing.IsActive = true;
    StoreProductResult queryResult = await
context.GetStoreProductForCurrentAppAsync();
    workingProgressRing.IsActive = false;

    if (queryResult.Product == null)
    {
        // The Store catalog returned an unexpected result.
        textBlock.Text = "Something went wrong, and the product was not
returned.";

        // Show additional error info if it is available.
        if (queryResult.ExtendedError != null)
        {
            textBlock.Text += $"{"\nExtendedError:
{queryResult.ExtendedError.Message}";
        }
    }
}
```

```

        return;
    }

    // Display the price of the app.
    textBlock.Text = $"The price of this app is:
{queryResult.Product.Price.FormattedBasePrice}";
}

```

## Get info for add-ons with known Store IDs that are associated with the current app

To get Store product info for add-ons that are associated with the current app and for which you already know the [Store IDs](#), use the [GetStoreProductsAsync](#) method. This is an asynchronous method that returns a collection of [StoreProduct](#) objects that represent each of the add-ons. In addition to the Store IDs, you must pass a list of strings to this method that identify the types of the add-ons. For a list of the supported string values, see the [ProductKind](#) property.

### ⓘ Note

The **GetStoreProductsAsync** method returns product info for the specified add-ons that are associated with the app, regardless of whether the add-ons are currently available for purchase. To retrieve info for all the add-ons for the current app that can currently be purchased, use the **GetAssociatedStoreProductsAsync** method as described in the [following section](#) instead.

This example retrieves info for durable add-ons with the specified Store IDs that are associated with the current app.

C#

```

private StoreContext context = null;

public async void GetProductInfo()
{
    if (context == null)
    {
        context = StoreContext.GetDefault();
        // If your app is a desktop app that uses the Desktop Bridge, you
        // may need additional code to configure the StoreContext object.
        // For more info, see https://aka.ms/storecontext-for-desktop.
    }

    // Specify the kinds of add-ons to retrieve.
    string[] productKinds = { "Durable" };
}

```

```

List<String> filterList = new List<string>(productKinds);

// Specify the Store IDs of the products to retrieve.
string[] storeIds = new string[] { "9NBLGGH4TNMP", "9NBLGGH4TNMN" };

workingProgressRing.IsActive = true;
StoreProductQueryResult queryResult =
    await context.GetStoreProductsAsync(filterList, storeIds);
workingProgressRing.IsActive = false;

if (queryResult.ExtendedError != null)
{
    // The user may be offline or there might be some other server
    failure.
    textBlock.Text = $"ExtendedError:
{queryResult.ExtendedError.Message}";
    return;
}

foreach (KeyValuePair<string, StoreProduct> item in
queryResult.Products)
{
    // Access the Store info for the product.
    StoreProduct product = item.Value;

    // Use members of the product object to access info for the
    product...
}
}

```

## Get info for add-ons that are available for purchase from the current app

To get Store product info for the add-ons that are currently available for purchase from the current app, use the [GetAssociatedStoreProductsAsync](#) method. This is an asynchronous method that returns a collection of [StoreProduct](#) objects that represent each of the available add-ons. You must pass a list of strings to this method that identify the types of add-ons you want to retrieve. For a list of the supported string values, see the [ProductKind](#) property.

### ⓘ Note

If the app has many add-ons that are available for purchase, you can alternatively use the [GetAssociatedStoreProductsWithPagingAsync](#) method to use paging to return the add-on results.

The following example retrieves info for all durable add-ons, Store-managed consumable add-ons, and developer-managed consumable add-ons that are available for purchase from the current app.

C#

```
private StoreContext context = null;

public async void GetAddOnInfo()
{
    if (context == null)
    {
        context = StoreContext.Default();
        // If your app is a desktop app that uses the Desktop Bridge, you
        // may need additional code to configure the StoreContext object.
        // For more info, see https://aka.ms/storecontext-for-desktop.
    }

    // Specify the kinds of add-ons to retrieve.
    string[] productKinds = { "Durable", "Consumable", "UnmanagedConsumable"
};
    List<String> filterList = new List<string>(productKinds);

    workingProgressRing.IsActive = true;
    StoreProductQueryResult queryResult = await
context.GetAssociatedStoreProductsAsync(filterList);
    workingProgressRing.IsActive = false;

    if (queryResult.ExtendedError != null)
    {
        // The user may be offline or there might be some other server
        failure.
        textBlock.Text = $"ExtendedError:
{queryResult.ExtendedError.Message}";
        return;
    }

    foreach (KeyValuePair<string, StoreProduct> item in
queryResult.Products)
    {
        // Access the Store product info for the add-on.
        StoreProduct product = item.Value;

        // Use members of the product object to access listing info for the
        add-on...
    }
}
```

## Get info for add-ons for the current app that the user has purchased

To get Store product info for add-ons that the current user has purchased, use the [GetUserCollectionAsync](#) method. This is an asynchronous method that returns a collection of [StoreProduct](#) objects that represent each of the add-ons. You must pass a list of strings to this method that identify the types of add-ons you want to retrieve. For a list of the supported string values, see the [ProductKind](#) property.

#### ⓘ Note

If the app has many add-ons, you can alternatively use the [GetUserCollectionWithPagingAsync](#) method to use paging to return the add-on results.

The following example retrieves info for durable add-ons with the specified [Store IDs](#).

C#

```
private StoreContext context = null;

public async void GetUserCollection()
{
    if (context == null)
    {
        context = StoreContext.Default();
        // If your app is a desktop app that uses the Desktop Bridge, you
        // may need additional code to configure the StoreContext object.
        // For more info, see https://aka.ms/storecontext-for-desktop.
    }

    // Specify the kinds of add-ons to retrieve.
    string[] productKinds = { "Durable" };
    List<String> filterList = new List<string>(productKinds);

    workingProgressRing.IsActive = true;
    StoreProductQueryResult queryResult = await
context.GetUserCollectionAsync(filterList);
    workingProgressRing.IsActive = false;

    if (queryResult.ExtendedError != null)
    {
        // The user may be offline or there might be some other server
        failure.
        textBlock.Text = $"ExtendedError:
{queryResult.ExtendedError.Message}";
        return;
    }

    foreach (KeyValuePair<string, StoreProduct> item in
queryResult.Products)
    {
        StoreProduct product = item.Value;
```

```
        // Use members of the product object to access info for the
product...
    }
}
```

## Related topics

- [In-app purchases and trials](#)
- [Get license info for apps and add-ons](#)
- [Enable in-app purchases of apps and add-ons](#)
- [Enable consumable add-on purchases](#)
- [Implement a trial version of your app](#)
- [Store sample](#) 

# Get license info for apps and add-ons

Article • 10/24/2022

This article demonstrates how to use methods of the [StoreContext](#) class in the [Windows.Services.Store](#) namespace to get license info for the current app and its add-ons. For example, you can use this info to determine if the licenses for the app or its add-ons are active, or if they are trial licenses.

## ⚠ Note

The **Windows.Services.Store** namespace was introduced in Windows 10, version 1607, and it can only be used in projects that target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio. If your app targets an earlier version of Windows 10, you must use the **Windows.ApplicationModel.Store** namespace instead of the **Windows.Services.Store** namespace. For more information, see [this article](#).

## Prerequisites

This example has the following prerequisites:

- A Visual Studio project for a Universal Windows Platform (UWP) app that targets **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release.
- You have [created an app submission](#) in Partner Center and this app is published in the Store. You can optionally configure the app so it is not discoverable in the Store while you test it. For more information, see our [testing guidance](#).
- If you want get license info for an add-on for the app, you must also [create the add-on in Partner Center](#).

The code in this example assumes:

- The code runs in the context of a [Page](#) that contains a [ProgressRing](#) named `workingProgressRing` and a [TextBlock](#) named `textBlock`. These objects are used to indicate that an asynchronous operation is occurring and to display output messages, respectively.
- The code file has a **using** statement for the **Windows.Services.Store** namespace.
- The app is a single-user app that runs only in the context of the user that launched the app. For more information, see [In-app purchases and trials](#).

### ⓘ Note

If you have a desktop application that uses the **Desktop Bridge**, you may need to add additional code not shown in this example to configure the **StoreContext** object. For more information, see [Using the StoreContext class in a desktop application that uses the Desktop Bridge](#).

## Code example

To get license info for the current app, use the [GetAppLicenseAsync](#) method. This is an asynchronous method that returns a [StoreAppLicense](#) object that provides license info for the app, including properties that indicate whether the user currently has a valid license to use the app ([IsActive](#)) and whether the license is for a trial version ([IsTrial](#)).

To access the licenses for durable add-ons of the current app for which the user has an entitlement to use, use the [AddOnLicenses](#) property of the [StoreAppLicense](#) object. This property returns a collection of [StoreLicense](#) objects that represent the add-on licenses.

```
C#

private StoreContext context = null;

public async void GetLicenseInfo()
{
    if (context == null)
    {
        context = StoreContext.Default();
        // If your app is a desktop app that uses the Desktop Bridge, you
        // may need additional code to configure the StoreContext object.
        // For more info, see https://aka.ms/storecontext-for-desktop.
    }

    workingProgressRing.IsActive = true;
    StoreAppLicense appLicense = await context.GetAppLicenseAsync();
    workingProgressRing.IsActive = false;

    if (appLicense == null)
    {
        textBlock.Text = "An error occurred while retrieving the license.";
        return;
    }

    // Use members of the appLicense object to access license info...

    // Access the valid licenses for durable add-ons for this app.
    foreach (KeyValuePair<string, StoreLicense> item in
        appLicense.AddOnLicenses)
```

```
{  
    StoreLicense addOnLicense = item.Value;  
    // Use members of the addOnLicense object to access license info  
    // for the add-on.  
}  
}
```

For a complete sample application, see the [Store sample](#).

## Related topics

- [In-app purchases and trials](#)
- [Get product info for apps and add-ons](#)
- [Enable in-app purchases of apps and add-ons](#)
- [Enable consumable add-on purchases](#)
- [Implement a trial version of your app](#)
- [Store sample](#)

# Enable in-app purchases of apps and add-ons

Article • 10/24/2022

This article demonstrates how to use members in the [Windows.Services.Store](#) namespace to request the purchase the current app or one of its add-ons for the user. For example, if the user currently has a trial version of the app, you can use this process to purchase a full license for the user. Alternatively, you can use this process to purchase an add-on, such as a new game level for the user.

To request the purchase of an app or add-on, the [Windows.Services.Store](#) namespace provides several different methods:

- If you know the [Store ID](#) of the app or add-on, you can use the [RequestPurchaseAsync](#) method of the [StoreContext](#) class.
- If you already have a [StoreProduct](#), [StoreSku](#), or [StoreAvailability](#) object that represents the app or add-on, you can use the [RequestPurchaseAsync](#) methods of these objects. For examples of different ways to retrieve a [StoreProduct](#) in your code, see [Get product info for apps and add-ons](#).

Each method presents a standard purchase UI to the user and then completes asynchronously after the transaction is complete. The method returns an object that indicates whether the transaction was successful.

## ⓘ Note

The **Windows.Services.Store** namespace was introduced in Windows 10, version 1607, and it can only be used in projects that target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio. If your app targets an earlier version of Windows 10, you must use the **Windows.ApplicationModel.Store** namespace instead of the **Windows.Services.Store** namespace. For more information, see [this article](#).

## Prerequisites

This example has the following prerequisites:

- A Visual Studio project for a Universal Windows Platform (UWP) app that targets **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release.

- You have [created an app submission](#) in Partner Center and this app is published in the Store. You can optionally configure the app so it is not discoverable in the Store while you test it. For more information, see our [testing guidance](#).
- If you want to enable in-app purchases for an add-on for the app, you must also [create the add-on in Partner Center](#).

The code in this example assumes:

- The code runs in the context of a [Page](#) that contains a [ProgressRing](#) named `workingProgressRing` and a [TextBlock](#) named `textBlock`. These objects are used to indicate that an asynchronous operation is occurring and to display output messages, respectively.
- The code file has a **using** statement for the **Windows.Services.Store** namespace.
- The app is a single-user app that runs only in the context of the user that launched the app. For more information, see [In-app purchases and trials](#).

#### ⓘ Note

If you have a desktop application that uses the **Desktop Bridge**, you may need to add additional code not shown in this example to configure the **StoreContext** object. For more information, see [Using the StoreContext class in a desktop application that uses the Desktop Bridge](#).

## Code example

This example demonstrates how to use the [RequestPurchaseAsync](#) method of the [StoreContext](#) class to purchase an app or add-on with a known [Store ID](#). For a complete sample application, see the [Store sample](#).

C#

```
private StoreContext context = null;

public async void PurchaseAddOn(string storeId)
{
    if (context == null)
    {
        context = StoreContext.Default();
        // If your app is a desktop app that uses the Desktop Bridge, you
        // may need additional code to configure the StoreContext object.
        // For more info, see https://aka.ms/storecontext-for-desktop.
    }

    workingProgressRing.IsActive = true;
```

```

StorePurchaseResult result = await
context.RequestPurchaseAsync(storeId);
workingProgressRing.IsActive = false;

// Capture the error message for the operation, if any.
string extendedError = string.Empty;
if (result.ExtendedError != null)
{
    extendedError = result.ExtendedError.Message;
}

switch (result.Status)
{
    case StorePurchaseStatus.AlreadyPurchased:
        textBlock.Text = "The user has already purchased the product.";
        break;

    case StorePurchaseStatus.Succeeded:
        textBlock.Text = "The purchase was successful.";
        break;

    case StorePurchaseStatus.NotPurchased:
        textBlock.Text = "The purchase did not complete. " +
            "The user may have cancelled the purchase. ExtendedError: "
+ extendedError;
        break;

    case StorePurchaseStatus.NetworkError:
        textBlock.Text = "The purchase was unsuccessful due to a network
error. " +
            "ExtendedError: " + extendedError;
        break;

    case StorePurchaseStatus.ServerError:
        textBlock.Text = "The purchase was unsuccessful due to a server
error. " +
            "ExtendedError: " + extendedError;
        break;

    default:
        textBlock.Text = "The purchase was unsuccessful due to an
unknown error. " +
            "ExtendedError: " + extendedError;
        break;
}
}

```

## Related topics

- [In-app purchases and trials](#)
- [Get product info for apps and add-ons](#)

- [Get license info for apps and add-ons](#)
- [Enable consumable add-on purchases](#)
- [Implement a trial version of your app](#)
- [Store sample](#) 

# Enable consumable add-on purchases

Article • 01/04/2023

This article demonstrates how to use methods of the [StoreContext](#) class in the [Windows.Services.Store](#) namespace to manage the user's fulfillment of consumable add-ons in your UWP apps. Use consumable add-ons for items that can be purchased, used, and purchased again. This is especially useful for things like in-game currency (gold, coins, etc.) that can be purchased and then used to purchase specific power-ups.

## ⓘ Note

The **Windows.Services.Store** namespace was introduced in Windows 10, version 1607, and it can only be used in projects that target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio. If your app targets an earlier version of Windows 10, you must use the **Windows.ApplicationModel.Store** namespace instead of the **Windows.Services.Store** namespace. For more information, see [this article](#).

## Overview of consumable add-ons

Apps can offer two types of consumable add-ons that differ in the way that fulfillments are managed:

- **Developer-managed consumable.** For this type of consumable, you are responsible for keeping track of the user's balance of items that the add-on represents, and for reporting the purchase of the add-on as fulfilled to the Store after the user has consumed all of the items. The user cannot purchase the add-on again until your app has reported the previous add-on purchase as fulfilled.

For example, if your add-on represents 100 coins in a game and the user consumes 10 coins, your app or service must maintain the new remaining balance of 90 coins for the user. After the user has consumed all 100 coins, your app must report the add-on as fulfilled, and then the user can purchase the 100 coin add-on again.

- **Store-managed consumable.** For this type of consumable, the Store keeps track of the user's balance of items that the add-on represents. When the user consumes any items, you are responsible for reporting those items as fulfilled to the Store, and the Store updates the user's balance. The user can purchase the add-on as many times as they want (they do not need to consume the items first). Your app can query the Store for the current balance for the user at any time.

For example, if your add-on represents an initial quantity of 100 coins in a game and the user consumes 50 coins, your app reports to the Store that 50 units of the add-on were fulfilled, and the Store updates the remaining balance. If the user then repurchases your add-on to acquire 100 more coins, they will now have 150 coins total.

#### ⓘ Note

Store-managed consumables were introduced in Windows 10, version 1607.

To offer a consumable add-on to a user, follow this general process:

1. Enable users to [purchase the add-on](#) from your app.
2. When the user consumes the add-on (for example, they spend coins in a game), [report the add-on as fulfilled](#).

At any time, you can also [get the remaining balance](#) for a Store-managed consumable.

## Prerequisites

These examples have the following prerequisites:

- A Visual Studio project for a Universal Windows Platform (UWP) app that targets **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release.
- You have [created an app submission](#) in Partner Center and this app is published in the Store. You can optionally configure the app so it is not discoverable in the Store while you test it. For more information, see our [testing guidance](#).
- You have [created a consumable add-on for the app](#) in Partner Center.

The code in these examples assume:

- The code runs in the context of a [Page](#) that contains a [ProgressRing](#) named `workingProgressRing` and a [TextBlock](#) named `textBlock`. These objects are used to indicate that an asynchronous operation is occurring and to display output messages, respectively.
- The code file has a `using` statement for the **Windows.Services.Store** namespace.
- The app is a single-user app that runs only in the context of the user that launched the app. For more information, see [In-app purchases and trials](#).

For a complete sample application, see the [Store sample](#).

#### ⓘ Note

If you have a desktop application that uses the **Desktop Bridge**, you may need to add additional code not shown in these examples to configure the **StoreContext** object. For more information, see [Using the StoreContext class in a desktop application that uses the Desktop Bridge](#).

## Report a consumable add-on as fulfilled

After the user [purchases the add-on](#) from your app and they consume your add-on, your app must report the add-on as fulfilled by calling the [ReportConsumableFulfillmentAsync](#) method of the **StoreContext** class. You must pass the following information to this method:

- The [Store ID](#) of the add-on that you want to report as fulfilled.
- The units of the add-on you want to report as fulfilled.
  - For a developer-managed consumable, specify 1 for the *quantity* parameter. This alerts the Store that the consumable has been fulfilled, and the customer can then purchase the consumable again. The user cannot purchase the consumable again until your app has notified the Store that it was fulfilled.
  - For a Store-managed consumable, specify the actual number of units that have been consumed. The Store will update the remaining balance for the consumable.
- The tracking ID for the fulfillment. This is a developer-supplied GUID that identifies the specific transaction that the fulfillment operation is associated with for tracking purposes. For more information, see the remarks in [ReportConsumableFulfillmentAsync](#).

This example demonstrates how to report a Store-managed consumable as fulfilled.

C#

```
private StoreContext context = null;

public async void ConsumeAddOn(string addOnStoreId)
{
    if (context == null)
    {
        context = StoreContext.Default();
        // If your app is a desktop app that uses the Desktop Bridge, you
        // may need additional code to configure the StoreContext object.
        // For more info, see https://aka.ms/storecontext-for-desktop.
    }

    // This is an example for a Store-managed consumable, where you specify
    // the actual number
    // of units that you want to report as consumed so the Store can update
```

```

the remaining
    // balance. For a developer-managed consumable where you maintain the
    balance, specify 1
    // to just report the add-on as fulfilled to the Store.
    uint quantity = 10;
    Guid trackingId = Guid.NewGuid();

    workingProgressRing.IsActive = true;
    StoreConsumableResult result = await
context.ReportConsumableFulfillmentAsync(
    addOnStoreId, quantity, trackingId);
    workingProgressRing.IsActive = false;

    // Capture the error message for the operation, if any.
    string extendedError = string.Empty;
    if (result.ExtendedError != null)
    {
        extendedError = result.ExtendedError.Message;
    }

    switch (result.Status)
    {
        case StoreConsumableStatus.Succeeded:
            textBlock.Text = "The fulfillment was successful. " +
                $"Remaining balance: {result.BalanceRemaining}";
            break;

        case StoreConsumableStatus.InsufficientQuantity:
            textBlock.Text = "The fulfillment was unsuccessful because the
remaining " +
                $"balance is insufficient. Remaining balance:
{result.BalanceRemaining}";
            break;

        case StoreConsumableStatus.NetworkError:
            textBlock.Text = "The fulfillment was unsuccessful due to a
network error. " +
                "ExtendedError: " + extendedError;
            break;

        case StoreConsumableStatus.ServerError:
            textBlock.Text = "The fulfillment was unsuccessful due to a
server error. " +
                "ExtendedError: " + extendedError;
            break;

        default:
            textBlock.Text = "The fulfillment was unsuccessful due to an
unknown error. " +
                "ExtendedError: " + extendedError;
            break;
    }
}

```

# Get the remaining balance for a Store-managed consumable

This example demonstrates how to use the [GetConsumableBalanceRemainingAsync](#) method of the [StoreContext](#) class to get the remaining balance for a Store-managed consumable add-on.

C#

```
private StoreContext context = null;

public async void GetRemainingBalance(string addOnStoreId)
{
    if (context == null)
    {
        context = StoreContext.GetDefault();
        // If your app is a desktop app that uses the Desktop Bridge, you
        // may need additional code to configure the StoreContext object.
        // For more info, see https://aka.ms/storecontext-for-desktop.
    }

    workingProgressRing.IsActive = true;
    StoreConsumableResult result = await
context.GetConsumableBalanceRemainingAsync(addOnStoreId);
    workingProgressRing.IsActive = false;

    // Capture the error message for the operation, if any.
    string extendedError = string.Empty;
    if (result.ExtendedError != null)
    {
        extendedError = result.ExtendedError.Message;
    }

    switch (result.Status)
    {
        case StoreConsumableStatus.Succeeded:
            textBlock.Text = "Remaining balance: " +
result.BalanceRemaining;
            break;

        case StoreConsumableStatus.NetworkError:
            textBlock.Text = "Could not retrieve balance due to a network
error. " +
                "ExtendedError: " + extendedError;
            break;

        case StoreConsumableStatus.ServerError:
            textBlock.Text = "Could not retrieve balance due to a server
error. " +
                "ExtendedError: " + extendedError;
            break;
    }
}
```

```
        default:
            textBlock.Text = "Could not retrieve balance due to an unknown
error. " +
                "ExtendedError: " + extendedError;
            break;
    }
}
```

## Related topics

- [In-app purchases and trials](#)
- [Get product info for apps and add-ons](#)
- [Get license info for apps and add-ons](#)
- [Enable in-app purchases of apps and add-ons](#)
- [Implement a trial version of your app](#)
- [Store sample](#) 

# Enable subscription add-ons for your app

Article • 01/04/2023

Your Universal Windows Platform (UWP) app can offer in-app purchases of *subscription* add-ons to your customers. You can use subscriptions to sell digital products in your app (such as app features or digital content) with automated recurring billing periods.

## ⓘ Note

To enable the purchase of subscription add-ons in your app, your project must target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio (this corresponds to Windows 10, version 1607), and it must use the APIs in the **Windows.Services.Store** namespace to implement the in-app purchase experience instead of the **Windows.ApplicationModel.Store** namespace. For more information about the differences between these namespaces, see [In-app purchases and trials](#).

## Feature highlights

Subscription add-ons for UWP apps support the following features:

- You can choose from subscription periods of 1 month, 3 months, 6 months, 1 year, or 2 years.
- You can add free trial periods of 1 week or 1 month to your subscription.
- The Windows SDK [provides APIs](#) you can use in your app to get info about available subscription add-ons for the app and enable the purchase of a subscription add-on. We also provide REST APIs you can call from your services to [manage subscriptions for a user](#).
- You can view analytic reports that provide the number of subscription acquisitions, active subscribers, and canceled subscriptions in a given time period.
- Customers can manage their subscription on the <https://account.microsoft.com/services> [↗](#) page for their Microsoft account. Customers can use this page to view all of the subscriptions they have acquired, cancel a subscription, and change the form of payment that is associated with their subscription.

# Steps to enable a subscription add-on for your app

To enable the purchase of subscription add-ons in your app, follow these steps.

1. [Create an add-on submission](#) for your subscription in Partner Center and publish the submission. As you follow the add-on submission process, pay close attention to the following properties:

- [Product type](#): Make sure you select **Subscription**.
- [Subscription period](#): Choose the recurring billing period for your subscription. You cannot change the subscription period after you publish your add-on.

Each subscription add-on supports a single subscription period and trial period. You must create a different subscription add-on for each type of subscription you want to offer in your app. For example, if you wanted to offer a monthly subscription with no trial, a monthly subscription with a one-month trial, an annual subscription with no trial, and an annual subscription with a one-month trial, you would need to create four subscription add-ons.

- [Trial period](#): Consider choosing a 1 week or 1 month trial period for your subscription to enable users to try your subscription content before they buy it. You cannot change or remove the trial period after you publish your subscription add-on.

To acquire a free trial of your subscription, a user must purchase your subscription through the standard in-app purchase process, including a valid form of payment. They are not charged any money during the trial period. At the end of the trial period, the subscription automatically converts to the full subscription and the user's payment instrument will be charged for the first period of the paid subscription. If the user chooses to cancel their subscription during the trial period, the subscription remains active until the end of the trial period. Some trial periods are not available for all subscription periods.

## ⓘ Note

Each customer can acquire a free trial for a subscription add-on only one time. After a customer acquires a free trial for a subscription, the Store prevents the same customer from ever acquiring the same free trial subscription again.

- **Visibility:** If you are creating a test add-on that you will only use to test the in-app purchase experience for your subscription, we recommend that you select one of the **Hidden in the Store** options. Otherwise, you can select the best visibility option for your scenario.
- **Pricing:** Choose the price of your subscription in this section. You cannot raise the price of the subscription after you publish the add-on. However, you can lower the price later.

#### Important

By default, when you create any add-on the price is initially set to **Free**. Because you cannot raise the price of a subscription add-on after you complete the add-on submission, be sure to choose the price of your subscription here.

2. In your app, use APIs in the **Windows.Services.Store** namespace to determine whether the current user has already acquired your subscription add-on and then offer it for sale to the user as an in-app purchase. See the [code examples](#) in this article for more details.
3. Test the in-app purchase implementation of your subscription in your app. You'll need to download your app once from the Store to your development device to use its license for testing. For more information, see our [testing guidance](#) for in-app purchases.
4. Create and publish an app submission that includes your updated app package, including your tested code. For more information, see [App submissions](#).

## Code examples

The code examples in this section demonstrate how to use the APIs in the **Windows.Services.Store** namespace to get info about subscription add-ons for the current app and request the purchase a subscription add-on on behalf of the current user.

These examples have the following prerequisites:

- A Visual Studio project for a Universal Windows Platform (UWP) app that targets **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release.
- You have [created an app submission](#) in Partner Center and this app is published in the Store. You can optionally configure the app so it is not discoverable in the

Store while you test it. For more information, see the [testing guidance](#).

- You have [created a subscription add-on for the app](#) in Partner Center.

The code in these examples assumes:

- The code file has **using** statements for the **Windows.Services.Store** and **System.Threading.Tasks** namespaces.
- The app is a single-user app that runs only in the context of the user that launched the app. For more information, see [In-app purchases and trials](#).

#### ⓘ Note

If you have a desktop application that uses the **Desktop Bridge**, you may need to add additional code not shown in these examples to configure the **StoreContext** object. For more information, see [Using the StoreContext class in a desktop application that uses the Desktop Bridge](#).

## Purchase a subscription add-on

This example demonstrates how to request the purchase of a known subscription add-on for your app on behalf of the current customer. This example also shows how to handle the case where the subscription has a trial period.

1. The code first determines whether the customer already has an active license for the subscription. If the customer already has an active license, your code should unlock the subscription features as necessary (because this is proprietary to your app, this is identified with a comment in the example).
2. Next, the code gets the **StoreProduct** object that represents the subscription you want to purchase on behalf of the customer. The code assumes that you already know the **Store ID** of the subscription add-on you want to purchase, and that you have assigned this value to the *subscriptionStoreId* variable.
3. The code then determines whether a trial is available for the subscription. Optionally, your app can use this information to display details about the available trial or full subscription to the customer.
4. Finally, the code calls **RequestPurchaseAsync** method to request the purchase of the subscription. If a trial is available for the subscription, the trial will be offered to the customer for purchase. Otherwise, the full subscription will be offered for purchase.

```

private StoreContext context = null;
StoreProduct subscriptionStoreProduct;

// Assign this variable to the Store ID of your subscription add-on.
private string subscriptionStoreId = "";

// This is the entry point method for the example.
public async Task SetupSubscriptionInfoAsync()
{
    if (context == null)
    {
        context = StoreContext.GetDefault();
        // If your app is a desktop app that uses the Desktop Bridge, you
        // may need additional code to configure the StoreContext object.
        // For more info, see https://aka.ms/storecontext-for-desktop.
    }

    bool userOwnsSubscription = await CheckIfUserHasSubscriptionAsync();
    if (userOwnsSubscription)
    {
        // Unlock all the subscription add-on features here.
        return;
    }

    // Get the StoreProduct that represents the subscription add-on.
    subscriptionStoreProduct = await GetSubscriptionProductAsync();
    if (subscriptionStoreProduct == null)
    {
        return;
    }

    // Check if the first SKU is a trial and notify the customer that a
    trial is available.
    // If a trial is available, the Skus array will always have 2
    purchasable SKUs and the
    // first one is the trial. Otherwise, this array will only have one SKU.
    StoreSku sku = subscriptionStoreProduct.Skus[0];
    if (sku.SubscriptionInfo.HasTrialPeriod)
    {
        // You can display the subscription trial info to the customer here.
        You can use
        // sku.SubscriptionInfo.TrialPeriod and
        sku.SubscriptionInfo.TrialPeriodUnit
        // to get the trial details.
    }
    else
    {
        // You can display the subscription purchase info to the customer
        here. You can use
        // sku.SubscriptionInfo.BillingPeriod and
        sku.SubscriptionInfo.BillingPeriodUnit
        // to provide the renewal details.
    }
}

```

```

        // Prompt the customer to purchase the subscription.
        await PromptUserToPurchaseAsync();
    }

    private async Task<bool> CheckIfUserHasSubscriptionAsync()
    {
        StoreAppLicense appLicense = await context.GetAppLicenseAsync();

        // Check if the customer has the rights to the subscription.
        foreach (var addOnLicense in appLicense.AddOnLicenses)
        {
            StoreLicense license = addOnLicense.Value;
            if (license.SkuStoreId.StartsWith(subscriptionStoreId))
            {
                if (license.IsActive)
                {
                    // The expiration date is available in the
                    license.ExpirationDate property.
                    return true;
                }
            }
        }

        // The customer does not have a license to the subscription.
        return false;
    }

    private async Task<StoreProduct> GetSubscriptionProductAsync()
    {
        // Load the sellable add-ons for this app and check if the trial is
        still
        // available for this customer. If they previously acquired a trial they
        won't
        // be able to get a trial again, and the StoreProduct.Skus property will
        // only contain one SKU.
        StoreProductQueryResult result =
            await context.GetAssociatedStoreProductsAsync(new string[] {
"Durable" });

        if (result.ExtendedError != null)
        {
            System.Diagnostics.Debug.WriteLine("Something went wrong while
getting the add-ons. " +
            "ExtendedError:" + result.ExtendedError);
            return null;
        }

        // Look for the product that represents the subscription.
        foreach (var item in result.Products)
        {
            StoreProduct product = item.Value;
            if (product.StoreId == subscriptionStoreId)
            {
                return product;
            }
        }
    }

```

```

    }

    System.Diagnostics.Debug.WriteLine("The subscription was not found.");
    return null;
}

private async Task PromptUserToPurchaseAsync()
{
    // Request a purchase of the subscription product. If a trial is
    // available it will be offered
    // to the customer. Otherwise, the non-trial SKU will be offered.
    StorePurchaseResult result = await
subscriptionStoreProduct.RequestPurchaseAsync();

    // Capture the error message for the operation, if any.
    string extendedError = string.Empty;
    if (result.ExtendedError != null)
    {
        extendedError = result.ExtendedError.Message;
    }

    switch (result.Status)
    {
        case StorePurchaseStatus.Succeeded:
            // Show a UI to acknowledge that the customer has purchased your
subscription
            // and unlock the features of the subscription.
            break;

        case StorePurchaseStatus.NotPurchased:
            System.Diagnostics.Debug.WriteLine("The purchase did not
complete. " +
                "The customer may have cancelled the purchase.
ExtendedError: " + extendedError);
            break;

        case StorePurchaseStatus.ServerError:
        case StorePurchaseStatus.NetworkError:
            System.Diagnostics.Debug.WriteLine("The purchase was
unsuccessful due to a server or network error. " +
                "ExtendedError: " + extendedError);
            break;

        case StorePurchaseStatus.AlreadyPurchased:
            System.Diagnostics.Debug.WriteLine("The customer already owns
this subscription." +
                "ExtendedError: " + extendedError);
            break;
    }
}

```

**Get info about subscription add-ons for the current app**

This code example demonstrates how to get info for all the subscription add-ons that are available in your app. To get this info, first use the [GetAssociatedStoreProductsAsync](#) method to get the collection of [StoreProduct](#) objects that represent each of the available add-ons for the app. Then, get the [StoreSku](#) for each product and use the [IsSubscription](#) and [SubscriptionInfo](#) properties to access the subscription info.

C#

```
private StoreContext context = null;

public async Task GetSubscriptionsInfo()
{
    if (context == null)
    {
        context = StoreContext.Default();
        // If your app is a desktop app that uses the Desktop Bridge, you
        // may need additional code to configure the StoreContext object.
        // For more info, see https://aka.ms/storecontext-for-desktop.
    }

    // Subscription add-ons are Durable products.
    string[] productKinds = { "Durable" };
    List<string> filterList = new List<string>(productKinds);

    StoreProductQueryResult queryResult =
        await context.GetAssociatedStoreProductsAsync(productKinds);

    if (queryResult.ExtendedError != null)
    {
        // The user may be offline or there might be some other server
        failure.
        System.Diagnostics.Debug.WriteLine($"ExtendedError:
{queryResult.ExtendedError.Message}");
        return;
    }

    foreach (KeyValuePair<string, StoreProduct> item in
queryResult.Products)
    {
        // Access the Store product info for the add-on.
        StoreProduct product = item.Value;

        // For each add-on, the subscription info is available in the SKU
        objects in the add-on.
        foreach (StoreSku sku in product.Skus)
        {
            if (sku.IsSubscription)
            {
                // Use the sku.SubscriptionInfo property to get info about
                the subscription.
                // For example, the following code gets the units and
```

```

duration of the
        // subscription billing period.
        StoreDurationUnit billingPeriodUnit =
sku.SubscriptionInfo.BillingPeriodUnit;
        uint billingPeriod = sku.SubscriptionInfo.BillingPeriod;
    }
}
}
}
}

```

## Manage subscriptions from your services

After your updated app is in the Store and customers can buy your subscription add-on, you may have scenarios where you need to manage the subscription for a customer. We provide REST APIs you can call from your services to perform the following subscription management tasks:

- [Get the subscriptions that a user has an entitlement to use](#). If your subscription is part of a cross-platform service, you can call this API to determine whether the specified user has an entitlement for your subscription and the status of their subscription in the context of your UWP app. You can then use this information to update the status of the subscription on other platforms that your service supports.
- [Change the billing state of a subscription for a given user](#). Use this API to cancel, extend, or disable automatic renewal for a subscription.

## Cancellations

Customers can use the <https://account.microsoft.com/services> page for their Microsoft account to view all of the subscriptions they have acquired, cancel a subscription, and change the form of payment that is associated with their subscription. When a customer cancels a subscription using this page, they continue to have access to the subscription for the duration of the current billing period. They do not get a refund for any part of the current billing period. At the end of the current billing period, their subscription is deactivated.

You can also cancel a subscription on behalf of a user by using our REST API to [change the billing state of a subscription for a given user](#).

## Subscription renewals and grace periods

At some point during each billing period, we will attempt to charge the customer's credit card for the next billing period. If the charge fails, the customer's subscription enters the *dunning* state. This means that their subscription is still active for the remainder of the current billing period, but we will periodically try to charge their credit card to automatically renew the subscription. This state can last up to two weeks, until the end of the current billing period and the renew date for the next billing period.

We do not offer grace periods for subscription billing. If we are unable to successfully charge the customer's credit card by the end of the current billing period, the subscription will be canceled and the customer will no longer have access to the subscription after the current billing period.

## Unsupported scenarios

The following scenarios are not currently supported for subscription add-ons.

- Selling subscriptions to customers directly via the Store is not supported at this time. Subscriptions are available for in-app purchases of digital products only.
- Customers cannot switch subscription periods using the <https://account.microsoft.com/services> page for their Microsoft account. To switch to a different subscription period, customers must cancel their current subscription and then purchase a subscription with a different subscription period from your app.
- Tier switching is currently not supported for subscription add-ons (for example, switching a customer from a basic subscription to a premium subscription with more features).
- [Sales](#) and [promotional codes](#) are currently not supported for subscription add-ons.
- Renewing existing subscriptions after setting the visibility of your subscription add-on to **Stop acquisition**. See [Set add-on pricing and availability](#) for more details.

## Related topics

- [In-app purchases and trials](#)
- [Get product info for apps and add-ons](#)

# Implement a trial version of your app

Article • 10/24/2022

If you [configure your app as a free trial in Partner Center](#) so that customers can use your app for free during a trial period, you can entice your customers to upgrade to the full version of your app by excluding or limiting some features during the trial period. Determine which features should be limited before you begin coding, then make sure that your app only allows them to work when a full license has been purchased. You can also enable features, such as banners or watermarks, that are shown only during the trial, before a customer buys your app.

This article shows how to use members of the [StoreContext](#) class in the [Windows.Services.Store](#) namespace to determine if the user has a trial license for your app and be notified if the state of the license changes while your app is running.

## ⓘ Note

The **Windows.Services.Store** namespace was introduced in Windows 10, version 1607, and it can only be used in projects that target **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio. If your app targets an earlier version of Windows 10, you must use the **Windows.ApplicationModel.Store** namespace instead of the **Windows.Services.Store** namespace. For more information, see [this article](#).

## Guidelines for implementing a trial version

The current license state of your app is stored as properties of the [StoreAppLicense](#) class. Typically, you put the functions that depend on the license state in a conditional block, as we describe in the next step. When considering these features, make sure you can implement them in a way that will work in all license states.

Also, decide how you want to handle changes to the app's license while the app is running. Your trial app can be full-featured, but have in-app ad banners where the paid-for version doesn't. Or, your trial app can disable certain features, or display regular messages asking the user to buy it.

Think about the type of app you're making and what a good trial or expiration strategy is for it. For a trial version of a game, a good strategy is to limit the amount of game content that a user can play. For a trial version of a utility, you might consider setting an expiration date, or limiting the features that a potential buyer can use.

For most non-gaming apps, setting an expiration date works well, because users can develop a good understanding of the complete app. Here are a few common expiration scenarios and your options for handling them.

- **Trial license expires while the app is running**

If the trial expires while your app is running, your app can:

- Do nothing.
- Display a message to your customer.
- Close.
- Prompt your customer to buy the app.

The best practice is to display a message with a prompt for buying the app, and if the customer buys it, continue with all features enabled. If the user decides not to buy the app, close it or remind them to buy the app at regular intervals.

- **Trial license expires before the app is launched**

If the trial expires before the user launches the app, your app won't launch. Instead, users see a dialog box that gives them the option to purchase your app from the Store.

- **Customer buys the app while it is running**

If the customer buys your app while it is running, here are some actions your app can take.

- Do nothing and let them continue in trial mode until they restart the app.
- Thank them for buying or display a message.
- Silently enable the features that are available with a full-license (or disable the trial-only notices).

Be sure to explain how your app will behave during and after the free trial period so your customers won't be surprised by your app's behavior. For more info about describing your app, see [Create app descriptions](#).

## Prerequisites

This example has the following prerequisites:

- A Visual Studio project for a Universal Windows Platform (UWP) app that targets **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release.
- You have created an app in Partner Center that is configured as a [free trial](#) with no time limit and this app is published in the Store. You can optionally configure the

app so it is not discoverable in the Store while you test it. For more information, see our [testing guidance](#).

The code in this example assumes:

- The code runs in the context of a [Page](#) that contains a [ProgressRing](#) named `workingProgressRing` and a [TextBlock](#) named `textBlock`. These objects are used to indicate that an asynchronous operation is occurring and to display output messages, respectively.
- The code file has a **using** statement for the **Windows.Services.Store** namespace.
- The app is a single-user app that runs only in the context of the user that launched the app. For more information, see [In-app purchases and trials](#).

#### ⓘ Note

If you have a desktop application that uses the **Desktop Bridge**, you may need to add additional code not shown in this example to configure the **StoreContext** object. For more information, see [Using the StoreContext class in a desktop application that uses the Desktop Bridge](#).

## Code example

When your app is initializing, get the [StoreAppLicense](#) object for your app and handle the [OfflineLicensesChanged](#) event to receive notifications when the license changes while the app is running. For example, the app's license could change if the trial period expires or the customer buys the app through a Store. When the license changes, get the new license and enable or disable a feature of your app accordingly.

At this point, if a user bought the app, it is a good practice to provide feedback to the user that the licensing status has changed. You might need to ask the user to restart the app if that's how you've coded it. But make this transition as seamless and painless as possible.

C#

```
private StoreContext context = null;
private StoreAppLicense appLicense = null;

// Call this while your app is initializing.
private async void InitializeLicense()
{
    if (context == null)
    {
```

```

        context = StoreContext.GetDefault();
        // If your app is a desktop app that uses the Desktop Bridge, you
        // may need additional code to configure the StoreContext object.
        // For more info, see https://aka.ms/storecontext-for-desktop.
    }

    workingProgressRing.IsActive = true;
    appLicense = await context.GetAppLicenseAsync();
    workingProgressRing.IsActive = false;

    // Register for the licenced changed event.
    context.OfflineLicensesChanged += context_OfflineLicensesChanged;
}

private async void context_OfflineLicensesChanged(StoreContext sender,
object args)
{
    // Reload the license.
    workingProgressRing.IsActive = true;
    appLicense = await context.GetAppLicenseAsync();
    workingProgressRing.IsActive = false;

    if (appLicense.IsActive)
    {
        if (appLicense.IsTrial)
        {
            textBlock.Text = $"This is the trial version. Expiration date:
{appLicense.ExpirationDate}";

            // Show the features that are available during trial only.
        }
        else
        {
            // Show the features that are available only with a full
license.
        }
    }
}
}

```

For a complete sample application, see the [Store sample](#).

## Related topics

- [In-app purchases and trials](#)
- [Get product info for apps and add-ons](#)
- [Get license info for apps and add-ons](#)
- [Enable in-app purchases of apps and add-ons](#)
- [Enable consumable add-on purchases](#)
- [Store sample](#)

# Data schemas for Store products

Article • 01/04/2023

When you submit a product such as an app or add-on to the Store, the Store maintains a comprehensive set of data for the product and its licenses. In your app's code, you can programmatically access some of this data by using properties in the [Windows.Services.Store](#) namespace. For example, you can retrieve the description and price of the current app or an add-on for the current app by using the [StoreProduct.Description](#) and [StoreProduct.Price](#) properties.

However, much of the data for products in the Store is not exposed by predefined properties in the [Windows.Services.Store](#) namespace. To access the complete data for a product in your code, you can use the following general properties instead:

- [StoreProduct.ExtendedJsonData](#)
- [StoreSku.ExtendedJsonData](#)
- [StoreAvailability.ExtendedJsonData](#)
- [StoreCollectionData.ExtendedJsonData](#)
- [StoreAppLicense.ExtendedJsonData](#)
- [StoreLicense.ExtendedJsonData](#)
- [StorePurchaseProperties.ExtendedJsonData](#)

These properties return the complete data for the corresponding object as a JSON-formatted string. This article provides the complete schema for the data returned by these properties.

## ⓘ Note

Products in the Store are organized in a hierarchy of **product**, **SKU**, and **availability** objects. For more information, see [Products, SKUs, and availabilities](#).

## Schema for StoreProduct, StoreSku, StoreAvailability, and StoreCollectionData

The following schema describes the JSON-formatted string returned by [StoreProduct.ExtendedJsonData](#). The [StoreSku.ExtendedJsonData](#), [StoreAvailability.ExtendedJsonData](#), and [StoreCollectionData.ExtendedJsonData](#) properties return only the portions of the schema that are defined under the

DisplaySkuAvailabilities\Sku, DisplaySkuAvailabilities\Availabilities, and DisplaySkuAvailabilities\Sku\CollectionData paths, respectively.

For an example of a JSON-formatted string returned by `StoreProduct.ExtendedJsonData`, see [this section](#).

JSON

```
{
  "type": "object",
  "properties": {
    "ProductId": {
      "type": "string",
      "pattern": "^[0-9A-Z]{12}$"
    },
    "ProductKind": {
      "type": "string"
    },
    "LocalizedProperties": {
      "type": "array",
      "minItems": 1,
      "maxItems": 1,
      "items": {
        "type": "object",
        "properties": {
          "DeveloperName": {
            "type": [ "string", "null" ]
          },
          "PublisherName": {
            "type": [ "string", "null" ]
          },
          "PublisherWebsiteUri": {
            "type": [ "string", "null" ]
          },
          "SupportUri": {
            "type": [ "string", "null" ]
          },
          "EligibilityProperties": {
            "type": [ "object", "null" ],
            "properties": {
              "Affirmations": {
                "type": "array",
                "items": {
                  "$ref": "#/definitions/affirmationDescription"
                }
              },
              "Remediations": {
                "type": "array",
                "items": {
                  "$ref": "#/definitions/remediationDescription"
                }
              }
            }
          }
        }
      }
    }
  },
}
```

```

        "additionalProperties": false
    },
    "ProductTitle": {
        "type": "string"
    },
    "ProductDescription": {
        "type": [ "string", "null" ]
    },
    "Language": {
        "type": "string"
    },
    "Images": {
        "type": [ "array", "null" ],
        "items": {
            "$ref": "#/definitions/image"
        }
    },
    "Videos": {
        "type": [ "array", "null" ],
        "items": {
            "$ref": "#/definitions/video"
        }
    },
    "SearchTitles": {
        "type": [ "array", "null" ],
        "items": {
            "type": "object",
            "properties": {
                "SearchTitleType": {
                    "type": "string"
                },
                "SearchTitleString": {
                    "type": "string"
                }
            }
        },
        "required": [
            "SearchTitleType",
            "SearchTitleString"
        ]
    }
},
"required": [
    "ProductTitle",
    "Language"
],
"additionalProperties": false
}
},
"MarketProperties": {
    "type": [ "array", "null" ],
    "minItems": 0,
    "maxItems": 1,
    "items": {
        "type": "object",

```

```

    "properties": {
      "RelatedProducts": {
        "type": [ "array", "null" ],
        "items": {
          "type": "object",
          "properties": {
            "RelationshipType": {
              "type": "string"
            },
            "RelatedProductId": {
              "type": "string",
              "pattern": "^[0-9A-Z]{12}$"
            }
          },
          "required": [
            "RelationshipType",
            "RelatedProductId"
          ]
        }
      },
      "additionalProperties": false
    },
    "Properties": {
      "type": [ "object", "null" ],
      "properties": {
        "InAppOfferToken": {
          "type": "string"
        },
        "PackageFamilyName": {
          "type": [
            "string",
            "null"
          ]
        },
        "Category": {
          "type": "string"
        },
        "IsAwardable": {
          "type": "boolean"
        },
        "IsColorizable": {
          "type": "boolean"
        },
        "Is3DExportable": {
          "type": "boolean"
        }
      },
      "additionalProperties": false
    },
    "DisplaySkuAvailabilities": {
      "type": "array",
      "minItems": 1,
      "items": {

```

```

        "$ref": "#/definitions/sku"
    }
}
},
"required": [
    "ProductId",
    "ProductKind",
    "LocalizedProperties",
    "DisplaySkuAvailabilities"
],
"format": "Product",
"definitions": {
    "sku": {
        "type": "object",
        "properties": {
            "Sku": {
                "type": "object",
                "properties": {
                    "SkuId": {
                        "type": "string",
                        "pattern": "^[0-9A-Z]{4}$"
                    },
                    "Properties": {
                        "type": "object",
                        "properties": {
                            "FulfillmentType": {
                                "type": [ "string", "null" ]
                            },
                            "CustomDeveloperData": {
                                "type": [ "string", "null" ]
                            },
                            "IsTrial": {
                                "type": [ "boolean", "null" ]
                            },
                            "SkuDisplayRank": {
                                "type": [ "integer", "null" ]
                            }
                        },
                        "BundledSkus": {
                            "type": [ "array", "null" ],
                            "items": {
                                "type": "object",
                                "properties": {
                                    "BigId": {
                                        "type": "string",
                                        "pattern": "^[0-9A-Z]{12}(?:/[0-9A-Z]{4})? $"
                                    }
                                },
                                "additionalProperties": false
                            }
                        },
                        "ConsumableQuantity": {
                            "type": "integer",
                            "minimum": 1,
                            "format": "ConsumableQuantity"
                        }
                    },

```

```
    "IsRepurchasable": {
      "type": "boolean"
    },
    "Packages": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "PackageUri": {
            "type": "string"
          }
        }
      },
      "required": [ "PackageUri" ],
      "additionalProperties": false
    }
  },
  "additionalProperties": false,
  "format": "SkuProperties"
},
"LocalizedProperties": {
  "type": "array",
  "minItems": 0,
  "maxItems": 1,
  "items": {
    "type": "object",
    "properties": {
      "SkuTitle": {
        "type": [ "string", "null" ]
      },
      "SkuDescription": {
        "type": [ "string", "null" ]
      },
      "SkuButtonTitle": {
        "type": [ "string", "null" ]
      },
      "Language": {
        "type": "string"
      }
    },
    "Images": {
      "type": [ "array", "null" ],
      "items": {
        "$ref": "#/definitions/image"
      }
    },
    "Videos": {
      "type": [ "array", "null" ],
      "items": {
        "$ref": "#/definitions/video"
      }
    },
    "LegalText": {
      "type": [ "object", "null" ],
      "properties": {
        "Tou": {
```

```

        "type": [ "string", "null" ]
    },
    "TouUri": {
        "type": [ "string", "null" ]
    },
    "Copyright": {
        "type": [ "string", "null" ]
    },
    "CopyrightUri": {
        "type": [ "string", "null" ]
    },
    "PrivacyPolicy": {
        "type": [ "string", "null" ]
    },
    "PrivacyPolicyUri": {
        "type": [ "string", "null" ]
    },
    "AdditionalLicenseTerms": {
        "type": [ "string", "null" ]
    }
},
"additionalProperties": false
},
"SkuDisplayRank": {
    "type": "array",
    "items": {
        "type": "object",
        "properties": {
            "Dimension": {
                "type": "string"
            },
            "Rank": {
                "type": "integer"
            }
        }
    },
    "additionalProperties": false
}
},
"required": [
    "Language"
],
"additionalProperties": false
}
},
"RecurrencePolicy": {
    "type": [
        "object",
        "null"
    ],
    "properties": {
        "InitialDuration": {
            "$ref": "#/definitions/duration"
        },
        "Duration": {

```

```

        "$ref": "#/definitions/duration"
    },
    "IsRecurring": {
        "type": [ "boolean", "null" ]
    },
    "HasTrial": {
        "type": [ "boolean", "null" ]
    }
},
"required": [ "InitialDuration", "Duration" ],
"additionalProperties": false
},
"CollectionData": {
    "$ref": "#/definitions/collectionData"
}
},
"required": [
    "SkuId",
    "Properties",
    "LocalizedProperties"
],
"additionalProperties": false
},
"Availabilities": {
    "type": "array",
    "minItems": 1,
    "items": {
        "$ref": "#/definitions/availability"
    }
}
},
"format": "SKU",
"required": [
    "Sku",
    "Availabilities"
],
"additionalProperties": false
},
"availability": {
    "type": "object",
    "properties": {
        "Conditions": {
            "type": "object",
            "properties": {
                "EndDate": {
                    "type": "string",
                    "pattern": "^\\d{4}-\\d\\d-\\d\\dT\\d\\d:\\d\\d:\\d\\d(\\.\\d+)?([+-]\\d\\d:\\d\\d\\d)|Z)?$"
                }
            },
            "required": [ "EndDate" ],
            "additionalProperties": false
        }
    },
    "required": [ "EndDate" ],
    "additionalProperties": false
},
"OrderManagementData": {
    "type": [ "object", "null" ],

```

```

    "properties": {
      "Price": {
        "type": "object",
        "properties": {
          "ListPrice": {
            "type": "number"
          },
          "CurrencyCode": {
            "enum": [ "AED", "AFN", "ALL", "AMD", "ANG", "AOA", "ARS",
"AUD", "AWG", "AZN", "BAM", "BBD", "BDT", "BGN", "BHD", "BIF", "BMD", "BND",
"BOB", "BOV", "BRL", "BSD", "BTN", "BWP", "BYR", "BZD", "CAD", "CDF", "CHE",
"CHF", "CHW", "CLF", "CLP", "CNY", "COP", "COU", "CRC", "CUC", "CUP", "CVE",
"CZK", "DJF", "DKK", "DOP", "DZD", "EGP", "ERN", "ETB", "EUR", "FJD", "FKP",
"GBP", "GEL", "GHS", "GIP", "GMD", "GNF", "GTQ", "GYD", "HKD", "HNL", "HRK",
"HTG", "HUF", "IDR", "ILS", "INR", "IQD", "IRR", "ISK", "JMD", "JOD", "JPY",
"KES", "KGS", "KHR", "KMF", "KPW", "KRW", "KWD", "KYD", "KZT", "LAK", "LBP",
"LRK", "LRD", "LSL", "LYD", "MAD", "MDL", "MGA", "MKD", "MMK", "MNT", "MOP",
"MRO", "MUR", "MVR", "MWK", "MXN", "MXV", "MYR", "MZN", "NAD", "NGN", "NIO",
"NOK", "NPR", "NZD", "OMR", "PAB", "PEN", "PGK", "PHP", "PKR", "PLN", "PYG",
"QAR", "RON", "RSD", "RUB", "RWF", "SAR", "SBD", "SCR", "SDG", "SEK", "SGD",
"SHP", "SLL", "SOS", "SRD", "SSP", "STD", "SYP", "SZL", "THB", "TJS", "TMT",
"TND", "TOP", "TRY", "TTD", "TWD", "TZS", "UAH", "UGX", "USD", "USN", "USS",
"UYI", "UYU", "UZS", "VEF", "VND", "VUV", "WST", "XAF", "XAG", "XAU", "XBA",
"XBB", "XBC", "XBD", "XCD", "XDR", "XFU", "XOF", "XPD", "XPF", "XPT", "XSU",
"XTS", "XUA", "XXX", "YER", "ZAR", "ZMW" ]
          },
          "MSRP": {
            "type": "number"
          },
          "RecurrencePrice": {
            "type": [ "number", "null" ]
          }
        },
        "required": [
          "ListPrice",
          "CurrencyCode",
          "MSRP"
        ],
        "additionalProperties": false
      }
    },
    "required": [ "Price" ],
    "additionalProperties": false
  },
  "AvailabilityId": {
    "type": "string",
    "pattern": "^[0-9A-Z]{12}$"
  },
  "DisplayRank": {
    "type": "integer"
  },
  "RemediationRequired": {
    "type": "boolean"
  },
  "Remediations": {

```

```
        "type": "array",
        "items": {
            "$ref": "#/definitions/remediation"
        }
    },
    "AffirmationId": {
        "type": "string"
    },
    "Actions": {
        "type": "array",
        "items": {
            "type": "string"
        }
    }
},
"format": "Availability",
"required": [
    "Conditions",
    "AvailabilityId",
    "DisplayRank",
    "Actions"
],
"additionalProperties": false
},
"duration": {
    "type": "object",
    "properties": {
        "UnitType": {
            "enum": [ "Year", "Month", "Week", "Day", "Hour", "Minute" ]
        },
        "Units": {
            "type": "integer"
        }
    }
},
"format": "Duration",
"required": [ "Units" ],
"additionalProperties": false
},
"collectionData": {
    "type": "object",
    "properties": {
        "productId": {
            "type": "string",
            "pattern": "^[0-9A-Z]{12}$"
        },
        "skuId": {
            "type": "string",
            "pattern": "^[0-9A-Z]{4}$"
        },
        "isTrial": {
            "type": [ "boolean", "null" ]
        },
        "campaignId": {
            "type": [ "string", "null" ]
        }
    },

```

```
    "devOfferId": {
      "type": [ "string", "null" ]
    },
    "acquiredDate": {
      "type": "string",
      "pattern": "^\\d{4}-\\d\\d-\\d\\dT\\d\\d:\\d\\d:\\d\\d(\\.\\d+)?((\\+|-)\\d\\d:\\d\\d)|Z)?$"
    },
    "endDate": {
      "type": "string",
      "pattern": "^\\d{4}-\\d\\d-\\d\\dT\\d\\d:\\d\\d:\\d\\d(\\.\\d+)?((\\+|-)\\d\\d:\\d\\d)|Z)?$"
    },
    "startDate": {
      "type": "string",
      "pattern": "^\\d{4}-\\d\\d-\\d\\dT\\d\\d:\\d\\d:\\d\\d(\\.\\d+)?((\\+|-)\\d\\d:\\d\\d)|Z)?$"
    },
    "modifiedDate": {
      "type": "string",
      "pattern": "^\\d{4}-\\d\\d-\\d\\dT\\d\\d:\\d\\d:\\d\\d(\\.\\d+)?((\\+|-)\\d\\d:\\d\\d)|Z)?$"
    },
    "autoRenew": {
      "type": "boolean"
    },
    "additionalIds": {
      "type": "array",
      "items": {
        "type": "object"
      }
    },
    "fulfillmentData": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "inAppOfferToken": {
      "type": "string"
    },
    "isCacheable": {
      "type": "boolean"
    },
    "itemId": {
      "type": "string"
    },
    "localTicketReference": {
      "type": "string"
    },
    "musicTracksInfo": {
      "type": "array",
      "items": {
        "type": "object"
      }
    }
  }
}
```

```
    },
    "orderId": {
      "type": "string",
      "pattern": "^[0-9A-Fa-f]{8}-[0-9A-Fa-f]{4}-[0-9A-Fa-f]{4}-[0-9A-Fa-f]{4}-[0-9A-Fa-f]{12}?$"
    },
    "orderLineItemId": {
      "type": "string",
      "pattern": "^[0-9A-Fa-f]{8}-[0-9A-Fa-f]{4}-[0-9A-Fa-f]{4}-[0-9A-Fa-f]{4}-[0-9A-Fa-f]{12}?$"
    },
    "ownershipType": {
      "type": "string"
    },
    "productFamily": {
      "type": "string"
    },
    "productKind": {
      "type": "string"
    },
    "productTitleId": {
      "type": "string"
    },
    "purchasedCountry": {
      "type": "string"
    },
    "quantity": {
      "type": "integer"
    },
    "skuType": {
      "type": "string"
    },
    "status": {
      "type": "string"
    },
    "tags": {
      "type": "array",
      "items": {
        "type": "string"
      }
    },
    "transactionId": {
      "type": "string",
      "pattern": "^[0-9A-Fa-f]{8}-[0-9A-Fa-f]{4}-[0-9A-Fa-f]{4}-[0-9A-Fa-f]{4}-[0-9A-Fa-f]{12}?$"
    }
  },
  "required": [
    "acquiredDate",
    "autoRenew",
    "additionalIds",
    "endDate",
    "fulfillmentData",
    "isCacheable",
    "itemId",
```

```

        "localTicketReference",
        "modifiedDate",
        "musicTracksInfo",
        "orderId",
        "orderLineItemId",
        "ownershipType",
        "productFamily",
        "productId",
        "productKind",
        "productTitleId",
        "purchasedCountry",
        "skuId",
        "skuType",
        "startDate",
        "status",
        "tags",
        "transactionId"
    ],
    "additionalProperties": false
},
"video": {
    "properties": {
        "Uri": {
            "type": "string"
        },
        "BackgroundColor": {
            "type": [
                "string",
                "null"
            ]
        },
        "ForegroundColor": {
            "type": [
                "string",
                "null"
            ]
        },
        "Caption": {
            "type": [
                "string",
                "null"
            ]
        },
        "Width": {
            "type": "integer"
        },
        "Height": {
            "type": "integer"
        },
        "VideoPurpose": {
            "type": [ "string", "null" ]
        },
        "PreviewImage": {
            "$ref": "#/definitions/image"
        }
    }
}

```

```

    },
    "required": [
        "Uri",
        "Width",
        "Height"
    ],
    "additionalProperties": false
},
"image": {
    "properties": {
        "Uri": {
            "type": "string"
        },
        "ImagePurpose": {
            "type": [
                "string",
                "null"
            ]
        },
        "BackgroundColor": {
            "type": [
                "string",
                "null"
            ]
        },
        "ForegroundColor": {
            "type": [
                "string",
                "null"
            ]
        },
        "Caption": {
            "type": [
                "string",
                "null"
            ]
        },
        "Width": {
            "type": "integer"
        },
        "Height": {
            "type": "integer"
        }
    },
    "required": [
        "Uri",
        "Width",
        "Height"
    ],
    "additionalProperties": false
},
"remediationDescription": {
    "type": "object",
    "properties": {
        "RemediationId": {

```

```

        "type": "string"
    },
    "Description": {
        "type": "string"
    }
},
"required": [ "RemediationId", "Description" ],
"additionalProperties": false
},
"remediation": {
    "type": "object",
    "properties": {
        "RemediationId": {
            "type": "string"
        },
        "Url": {
            "type": "string"
        },
        "BigId": {
            "type": "string",
            "pattern": "^[0-9A-Z]{12}(?:/[0-9A-Z]{4})?$"
        }
    },
    "required": [
        "RemediationId"
    ],
    "additionalProperties": false
},
"affirmationDescription": {
    "type": "object",
    "properties": {
        "AffirmationId": {
            "type": "string"
        },
        "AffirmationProductId": {
            "type": [ "string", "null" ],
            "pattern": "^[0-9A-Z]{12}$"
        },
        "Description": {
            "type": "string"
        }
    },
    "required": [ "AffirmationId", "Description" ],
    "additionalProperties": false
}
},
"additionalProperties": false
}

```

## Example

The following example demonstrates a JSON-formatted string returned by the `StoreProduct.ExtendedJsonData` property for app.

#### JSON

```
{
  "LocalizedProperties": [
    {
      "DeveloperName": "Contoso developer",
      "PublisherName": "Contoso",
      "PublisherWebsiteUri": "http://www.contoso.com",
      "SupportUri": "mailto:support@contoso.com",
      "EligibilityProperties": null,
      "Images": [
        {
          "BackgroundColor": "transparent",
          "Caption": "",
          "ForegroundColor": "",
          "Height": 300,
          "ImagePurpose": "tile",
          "Uri": "//store-images.s-contoso.com/image/apps.40735.14525693010133175.0aff34d4-9b25-4671-b5fc-0d13efd28147.c23ea293-a138-42a0-a615-d03c41409169",
          "Width": 300
        },
        {
          "BackgroundColor": "transparent",
          "Caption": "",
          "ForegroundColor": "",
          "Height": 88,
          "ImagePurpose": "tile",
          "Uri": "//store-images.s-contoso.com/image/apps.25013.14525693010133175.40f2a63d-463b-49f6-8953-b2d9cc4abeba.baa4030d-5d37-4965-8039-72158df22c42",
          "Width": 88
        },
        {
          "BackgroundColor": "transparent",
          "Caption": "",
          "ForegroundColor": "",
          "Height": 300,
          "ImagePurpose": "tile",
          "Uri": "//store-images.s-contoso.com/image/apps.61404.14525693010133175.b9fde0dd-bab8-49a1-acfb-35d897b02bda.c0a07eff-ce1b-47c8-a4b5-5c83337911f9",
          "Width": 620
        },
        {
          "BackgroundColor": "transparent",
          "Caption": "",
          "ForegroundColor": "",
          "Height": 50,
          "ImagePurpose": "logo",
          "Uri": "//store-images.s-
```

```

contoso.com/image/apps.53845.14525693010133175.f907b7fa-6bd3-4d9e-af7d-
33f19e170d33.07b372ce-4514-4f71-832f-745cad63c78e",
    "Width": 50
  },
  {
    "BackgroundColor": "transparent",
    "Caption": "",
    "ForegroundColor": "",
    "Height": 300,
    "ImagePurpose": "Logo",
    "Uri": "//store-images.s-
contoso.com/image/apps.27529.14525693010133175.f7574c79-bdda-4684-9929-
25f7e1d370b7.a16a54ee-c3b1-4311-9d36-5fee7ac23639",
    "Width": 300
  },
  {
    "BackgroundColor": "transparent",
    "Caption": "",
    "ForegroundColor": "",
    "Height": 768,
    "ImagePurpose": "Screenshot",
    "Uri": "//store-images.s-
contoso.com/image/apps.39690.14525693010133175.47a548ec-c651-4b73-b937-
7953a8714ddd.b2724431-1470-419e-aa4b-74830aba9e1b",
    "Width": 1366
  }
],
"Videos": [],
"ProductDescription": "This is a sample app for developement tasks",
"ProductTitle": "Contoso Sample App",
"SearchTitles": [],
"Language": "en-us"
}
],
"MarketProperties": [
  {
    "RelatedProducts": []
  }
],
"ProductId": "9NBLGGH4R315",
"Properties": {
  "PackageFamilyName": "30321Contoso.ContosoSampleApp_sv9ybgxvtektj"
},
"ProductKind": "Application",
"DisplaySkuAvailabilities": [
  {
    "Sku": {
      "LocalizedProperties": [
        {
          "LegalText": {
            "Copyright": "",
            "CopyrightUri": "",
            "PrivacyPolicy": "",
            "PrivacyPolicyUri": "",
            "Tou": "",

```

```
        "TouUri": ""
    },
    "SkuDescription": "This is a sample app for developement tasks",
    "SkuTitle": "Contoso Sample App",
    "SkuDisplayRank": [],
    "Language": "en-us"
}
],
"Properties": {
    "FulfillmentType": "WindowsUpdate",
    "BundledSkus": [],
    "IsRepurchasable": false,
    "IsTrial": false
},
"SkuId": "0010",
"RecurrencePolicy": null,
"CollectionData": {
    "acquiredDate": "2017-02-27T13:34:57.6680551-08:00",
    "additionalIds": [],
    "beneficiary": {
        "identityType": "msa",
        "identityValue": "949157616400113"
    },
    "endDate": "9999-12-31T15:59:59.9999999-08:00",
    "fulfillmentData": [],
    "isCacheable": true,
    "itemId": "0ae08fb4326a413a890c957b52aeda88",
    "localTicketReference": "0",
    "modifiedDate": "2017-02-27T13:34:57.6616413-08:00",
    "musicTracksInfo": [],
    "orderId": "9c54da93-5b1d-4afe-ac45-dd0c19bb3e8f",
    "orderLineItemId": "88043dae-9cca-4497-aaac-e85f6cf018aa",
    "ownershipType": "OwnedByBeneficiary",
    "productFamily": "Apps",
    "productId": "9NBLGGH4R315",
    "productKind": "Application",
    "productTitleId": "-129071",
    "purchasedCountry": "US",
    "purchaser": {
        "identityType": "msa",
        "identityValue": "985157600400620"
    },
    "quantity": 1,
    "skuId": "0010",
    "skuType": "Full",
    "startDate": "2017-02-27T13:19:57.6680551-08:00",
    "status": "Active",
    "tags": [],
    "transactionId": "9c54da93-5b1d-4afe-ac45-dd0c19bb3e8f"
}
},
"Availabilities": [
{
    "Actions": [
        "Details",
```

```

        "Fulfill",
        "License",
        "Purchase",
        "Redeem"
    ],
    "AvailabilityId": "9XJKQMZ5M9NX",
    "Conditions": {
        "EndDate": "9998-12-30T00:00:00Z"
    },
    "OrderManagementData": {
        "Price": {
            "CurrencyCode": "USD",
            "ListPrice": 0,
            "MSRP": 0
        }
    },
    "DisplayRank": 0,
    "RemediationRequired": false
}
]
},
{
    "Sku": {
        "LocalizedProperties": [
            {
                "LegalText": {
                    "Copyright": "",
                    "CopyrightUri": "",
                    "PrivacyPolicy": "",
                    "PrivacyPolicyUri": "",
                    "Tou": "",
                    "TouUri": ""
                },
                "SkuDescription": "This is a sample app for developement tasks",
                "SkuTitle": "Contoso Sample App",
                "SkuDisplayRank": [],
                "Language": "en-us"
            }
        ],
        "Properties": {
            "FulfillmentType": "WindowsUpdate",
            "BundledSkus": [],
            "IsRepurchasable": false,
            "IsTrial": false
        },
        "SkuId": "0017",
        "RecurrencePolicy": null
    },
    "Availabilities": [
        {
            "Actions": [
                "Details"
            ],
            "AvailabilityId": "9ZFNZ00M33HF",
            "Conditions": {

```

```

        "EndDate": "9998-12-30T00:00:00Z"
    },
    "OrderManagementData": {
        "Price": {
            "CurrencyCode": "USD",
            "ListPrice": 0,
            "MSRP": 0
        }
    },
    "DisplayRank": 0,
    "RemediationRequired": false
}
]
},
{
    "Sku": {
        "LocalizedProperties": [
            {
                "LegalText": {
                    "Copyright": "",
                    "CopyrightUri": "",
                    "PrivacyPolicy": "",
                    "PrivacyPolicyUri": "",
                    "Tou": "",
                    "TouUri": ""
                },
                "SkuDescription": "This is a sample app for developement tasks",
                "SkuTitle": "Contoso Sample App",
                "SkuDisplayRank": [],
                "Language": "en-us"
            }
        ],
        "Properties": {
            "FulfillmentType": "WindowsUpdate",
            "BundledSkus": [],
            "IsRepurchasable": false,
            "IsTrial": true
        },
        "SkuId": "0011",
        "RecurrencePolicy": null
    },
    "Availabilities": [
        {
            "Actions": [
                "Details",
                "License",
                "Fulfill"
            ],
            "AvailabilityId": "9QVHW1D0B4QB",
            "Conditions": {
                "EndDate": "9998-12-30T00:00:00Z"
            },
            "OrderManagementData": {
                "Price": {
                    "CurrencyCode": "USD",

```

```

        "ListPrice": 0,
        "MSRP": 0
    }
},
"DisplayRank": 0,
"RemediationRequired": false
}
]
}
]
}

```

## Schema for StoreAppLicense and StoreLicense

The following schema describes the JSON-formatted string returned by [StoreAppLicense.ExtendedJsonData](#). The [StoreLicense.ExtendedJsonData](#) property returns only the portions of the schema that are defined under the `productAddOns` path.

For an example of a JSON-formatted string returned by [StoreAppLicense.ExtendedJsonData](#), see [this section](#).

JSON

```

{
  "type": "object",
  "properties": {
    "productId": {
      "type": "string",
      "pattern": "^[0-9A-Z]{12}$"
    },
    "skuId": {
      "type": "string",
      "pattern": "^[0-9A-Z]{4}$"
    },
    "expiration": {
      "type": "string",
      "pattern": "^\\d{4}-\\d\\d-\\d\\d \\d\\d:\\d\\d:\\d\\d(\\.\\d+)?$"
    },
    "isActive": {
      "type": "boolean"
    },
    "isTrial": {
      "type": "boolean"
    },
    "isTrialOwnedByThisUser": {
      "type": "boolean"
    },
    "trialTimeRemaining": {
      "type": "string"
    }
  }
}

```

```

    },
    "productAddOns": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "inAppOfferToken": {
            "type": "string"
          },
          "productId": {
            "type": "string",
            "pattern": "^[0-9A-Z]{12}$"
          },
          "productType": {
            "type": "string"
          },
          "skuId": {
            "type": "string",
            "pattern": "^[0-9A-Z]{4}$"
          },
          "skuType": {
            "type": "string"
          },
          "expiration": {
            "type": "string",
            "pattern": "^\\d{4}-\\d\\d-\\d\\d\\d\\d\\d:\\d\\d:\\d\\d(\\.\\d+)?$"
          },
          "isActive": {
            "type": "boolean"
          }
        }
      },
      "required": [
        "inAppOfferToken",
        "productType",
        "skuType",
        "expiration",
        "isActive"
      ],
      "additionalProperties": false
    }
  },
  "required": [
    "productAddOns",
    "productId",
    "skuId",
    "expiration",
    "isActive",
    "isTrial",
    "isTrialOwnedByThisUser",
    "trialTimeRemaining"
  ],
  "additionalProperties": false
}

```

## Example

The following example demonstrates a JSON-formatted string returned by the [StoreAppLicense.ExtendedJsonData](#) property for app.

JSON

```
{
  "productAddOns": [
    {
      "inAppOfferToken": "Durable test add-on",
      "productId": "9NBLGGH4TNMP",
      "productType": "Durable",
      "skuId": "0010",
      "skuType": "Full",
      "expiration": "9999-12-31 00:00:00",
      "isActive": true
    },
    {
      "inAppOfferToken": "Consumable test add-on",
      "productId": "9NBLGGH4TNMN",
      "productType": "Consumable",
      "skuId": "0020",
      "skuType": "Full",
      "expiration": "9999-12-31 00:00:00",
      "isActive": true
    }
  ],
  "productId": "9NBLGGH4R315",
  "skuId": "0010",
  "isActive": true,
  "isTrial": false,
  "isTrialOwnedByThisUser": false,
  "expiration": "9999-12-31 00:00:00"
}
```

## Schema for StorePurchaseProperties

The following schema describes the JSON-formatted string returned by [StorePurchaseProperties.ExtendedJsonData](#).

JSON

```
{
  "type": "object",
  "properties": {
    "Name": {
```

```
        "type": "string"
      },
    },
    "required": [
      "Name"
    ],
    "additionalProperties": false
  }
}
```

## Related topics

- [In-app purchases and trials](#)
- [Get product info for apps and add-ons](#)
- [Get license info for apps and add-ons](#)
- [Enable in-app purchases of apps and add-ons](#)
- [Enable consumable add-on purchases](#)
- [Implement a trial version of your app](#)

# Error codes for Store operations

Article • 03/07/2024

This article describes common error codes that you might encounter while you are developing or testing Store-related operations in your app.

## In-app purchase error codes

The following error codes are related to in-app purchase operations.

 Expand table

| Error code | Description                                                                                                                                                                                                                                            |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x803F6100 | The in-app purchase could not be completed because Kid's Corner is active. To complete the purchase, sign in to the device with your Microsoft account and run the application again.                                                                  |
| 0x803F6101 | The specified app could not be found. The app may no longer be available in the Store, or you might have provided the wrong Store ID for the app.                                                                                                      |
| 0x803F6102 | The specified add-on could not be found. The add-on may no longer be available in the Store, or you might have provided the wrong Store ID for the add-on.                                                                                             |
| 0x803F6103 | The specified product could not be found. The product may no longer be available in the Store, or you might have provided the wrong Store ID for the product.                                                                                          |
| 0x803F6104 | The in-app purchase could not be completed because you are running a trial version of the app. To complete in-app purchases, install the full version of the app.                                                                                      |
| 0x803F6105 | The in-app purchase could not be completed because you are not signed in with your Microsoft account.                                                                                                                                                  |
| 0x803F6107 | Something unexpected happened while processing the current operation.                                                                                                                                                                                  |
| 0x803F6108 | The in-app purchase could not be completed because the app license is missing information. This error can occur when you side-load your app. To resolve this issue, uninstall the app and then reinstall it from the Store to refresh the app license. |
| 0x803F6109 | The consumable add-on fulfillment could not be completed because the specified quantity is more than the remaining balance.                                                                                                                            |
| 0x803F610A | The specified provider type for the Store user account is not supported.                                                                                                                                                                               |
| 0x803F610B | The specified Store operation is not supported.                                                                                                                                                                                                        |

| Error code | Description                                                      |
|------------|------------------------------------------------------------------|
| 0x803F610C | The app does not support the specified background task contract. |
| 0x80040001 | The provided list of add-on product IDs is invalid.              |
| 0x80040002 | The provided list of keywords is invalid.                        |
| 0x80040003 | The fulfillment target is invalid.                               |

## Licensing error codes

The following error codes are related to licensing operations for apps or add-ons.

 Expand table

| Error code                                                         | Description                                                                                                                                                                                       |
|--------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0x803F700C                                                         | The device is currently offline. To use this app while the device is offline, open your Store settings and toggle the <b>Offline Permissions</b> setting.                                         |
| 0x803F8001                                                         | You do not have an entitlement for the product. You might be using a different Microsoft account than the one that was used to purchase the product.                                              |
| 0x803F8002                                                         | Your entitlement for the product has expired.                                                                                                                                                     |
| 0x803F8003                                                         | Your entitlement for the product is in an invalid state that prevents a license from being created.                                                                                               |
| 0x803F8009<br>0x803F800A                                           | The trial period for the app has expired.                                                                                                                                                         |
| 0x803F8190                                                         | The license doesn't allow the product to be used in the current country or region of your device.                                                                                                 |
| 0x803F81F5<br>0x803F81F6<br>0x803F81F7<br>0x803F81F8<br>0x803F81F9 | You have reached the maximum number of devices that can be used with games and apps from the Store. To use this game or app on the current device, first remove another device from your account. |
| 0x803F9000<br>0x803F9001                                           | The license is expired or corrupt. To help resolve this error, try running the <a href="#">troubleshooter for Windows apps</a> to reset the Store cache.                                          |
| 0x803F9006                                                         | The operation could not be completed because the user who is entitled to this product is not signed in to the device with their Microsoft account.                                                |
| 0x803F9008<br>0x803F9009                                           | Your device is offline. Your device needs to be online to use this product.                                                                                                                       |

| Error code | Description                   |
|------------|-------------------------------|
| 0x803F900A | The subscription has expired. |

## Self-install update error codes

The following error codes are related to [self-installing package updates](#).

 Expand table

| Error code | Description                                                                                                                          |
|------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 0x803F6200 | User consent is required to download the package update.                                                                             |
| 0x803F6201 | User consent is required to download and install the package update.                                                                 |
| 0x803F6203 | User consent is required to install the package update.                                                                              |
| 0x803F6204 | User consent is required to download the package update because the download will occur on a metered network connection.             |
| 0x803F6206 | User consent is required to download and install the package update because the download will occur on a metered network connection. |

## Related topics

- [In-app purchases and trials](#)
- [Get product info for apps and add-ons](#)
- [Get license info for apps and add-ons](#)
- [Enable in-app purchases of apps and add-ons](#)
- [Enable consumable add-on purchases](#)
- [Enable subscription add-ons for your app](#)
- [Implement a trial version of your app](#)

---

## Feedback

Was this page helpful?

 Yes

 No

[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

# In-app purchases and trials using the `Windows.ApplicationModel.Store` namespace

Article • 01/04/2023

You can use members in the [Windows.ApplicationModel.Store](#) namespace to add in-app purchases and trial functionality to your Universal Windows Platform (UWP) app to help monetize your app. These APIs also provide access to the license info for your app.

The articles in this section provide in-depth guidance and code examples for using the members in the **Windows.ApplicationModel.Store** namespace for several common scenarios. For an overview of basic concepts related to in-app purchases in UWP apps, see [In-app purchases and trials](#). For a complete sample that demonstrates how to implement trials and in-app purchases using the **Windows.ApplicationModel.Store** namespace, see the [Store sample](#) .

## Important

The **Windows.ApplicationModel.Store** namespace is no longer being updated with new features. If your project targets **Windows 10 Anniversary Edition (10.0; Build 14393)** or a later release in Visual Studio (that is, you are targeting Windows 10, version 1607, or later), we recommend that you use the [Windows.Services.Store](#) namespace instead. For more information, see [In-app purchases and trials](#). The **Windows.ApplicationModel.Store** namespace is not supported in Windows desktop applications that use the [Desktop Bridge](#) or in apps or games that use a development sandbox in Partner Center (for example, this is the case for any game that integrates with Xbox Live). These products must use the **Windows.Services.Store** namespace to implement in-app purchases and trials.

## Get started with the `CurrentApp` and `CurrentAppSimulator` classes

The main entry point to the **Windows.ApplicationModel.Store** namespace is the [CurrentApp](#) class. This class provides static properties and methods you can use to get info for the current app and its available add-ons, get license info for the current app or its add-ons, purchase an app or add-on for the current user, and perform other tasks.